

Soutenance LO07 – BlaBlaCar 2026

Partie de Pierre : la couche **Vue** et le visuel

Pierre Bheidi (binôme : Moss'Ab Mirande-Ney)

1 Ce que je présente

Je suis responsable de la **couche Vue** de l'architecture MVC : tout l'affichage, la barre de menu dynamique, les formulaires, les tableaux, et l'intégration **Bootstrap 5**. Mon rôle est de transformer les données préparées par les contrôleurs en pages HTML lisibles, sans jamais y mettre de logique métier.

2 Texte à dire pendant la démo

2.1 1. Rôle de la Vue dans le MVC

« Dans le pattern MVC, la Vue ne s'occupe que de l'**affichage**. Elle ne fait aucune requête à la base et aucun calcul métier : elle reçoit des données déjà prêtes du contrôleur et les met en forme. Cette séparation rend le code clair et réutilisable. »

2.2 2. Organisation des fichiers de vue

« Les vues sont rangées par rôle dans `app/views/` : `admin/`, `conducteur/`, `passager/`, `examineur/`, `innovations/`, plus un dossier `layout/` commun qui contient l'en-tête, le pied de page et la barre de menu. »

2.3 3. Le gabarit commun et le rendu

« Chaque page est composée automatiquement : le contrôleur appelle une méthode `render()` qui assemble **header** + **fragmentMenu** + **la vue** + **footer**. On n'écrit donc jamais deux fois le `<head>` ou la barre de menu. »

```
echo '<main class="container flex-grow-1 pb-4">';
require $contenuVue;           // la vue de la page
echo '</main>';
```

2.4 4. La barre de menu dynamique (fragmentMenu)

« C'est la pièce centrale de la Vue. Elle lit l'utilisateur connecté en session et affiche **un seul** menu métier selon son rôle (Administrateur, Conducteur ou Passager), plus les menus toujours visibles : Innovations, Examineur, Se connecter. Si personne n'est connecté, aucun menu métier n'apparaît. »

```
if ($courant !== null && $courant['role'] === 'administrateur'): ...
elseif ($courant['role'] === 'conducteur'): ...
elseif ($courant['role'] === 'passager'): ...
```

« On affiche aussi en haut à gauche les noms du binôme, le nom de l'utilisateur connecté et son solde – comme demandé dans le sujet. »

2.5 5. Bootstrap 5, zéro CSS maison

« Toute la mise en forme repose sur **Bootstrap 5** chargé par CDN : pas de fichier CSS personnel à maintenir. J'utilise les composants : **navbar** pour la barre, **dropdown** pour les menus déroulants, **card** pour les formulaires et les statistiques, **table table-striped** pour les listes, **badge** pour les statuts, **alert** pour les messages. »

2.6 6. Formulaires et tableaux

« Les formulaires (connexion, ajout d'un conducteur, création d'un trajet) utilisent **form-control** et **form-select**. Les listes sont des tableaux à en-tête sombre. Les statuts de trajet sont des badges verts (actif) ou gris (passif). »

2.7 7. Sécurité côté Vue

« Toutes les données affichées sont échappées avec **htmlspecialchars** pour éviter les failles XSS : si un nom contient du code HTML, il s'affiche en texte et ne s'exécute pas. »

2.8 8. Messages à l'utilisateur (flash)

« Après une action (ajout réussi, réservation confirmée), un message de confirmation s'affiche en vert ; en cas d'erreur, en rouge. »

3 Ce que je montre à l'écran

1. La page d'accueil et la barre de menu sans connexion (seuls 3 menus visibles).
2. Connexion : le menu change selon le rôle (effet « dynamique »).
3. Une liste (utilisateurs ou véhicules) : tableau Bootstrap, propriétaire reconstruit prénom + nom (pas de clé primaire affichée).
4. Un formulaire d'ajout et le message de confirmation.
5. Le responsive : réduire la fenêtre, la barre se replie (menu hamburger).

4 Questions potentielles du jury (et réponses)

Q : Pourquoi séparer la Vue du reste du code ?

R : Pour respecter le principe de séparation des responsabilités : on peut changer l'affichage (passer à Bootstrap par exemple) sans toucher à la logique, et réutiliser un même contrôleur avec plusieurs vues.

Q : Comment la barre de menu sait quel menu afficher ?

R : Elle lit `$_SESSION['login_id']`, récupère l'utilisateur via le modèle, puis teste son champ `role`. Un seul des trois menus métier est rendu selon ce rôle.

Q : Que se passe-t-il si aucun utilisateur n'est connecté ?

R : `login_id` vaut -1, donc aucun menu métier n'est affiché : seuls Innovations, Examineur et Se connecter restent visibles, comme l'exige le sujet.

Q : À quoi sert `htmlspecialchars` ?

R : À échapper les caractères spéciaux HTML en sortie. C'est une protection contre les attaques XSS : du code injecté dans un nom s'affiche comme du texte au lieu d'être exécuté par le navigateur.

Q : Pourquoi Bootstrap plutôt qu'un CSS écrit à la main ?

R : Bootstrap fournit des composants testés et un rendu responsive immédiat, ce qui nous a permis de nous concentrer sur la logique. La structure HTML reste la nôtre ; on maîtrise les classes utilisées.

Q : Comment une vue reçoit-elle ses données ?

R : Le contrôleur passe un tableau associatif à `render()`, qui fait un `extract()` : chaque clé devient une variable disponible dans la vue (par exemple `$utilisateurs`).

Q : Différence entre `<?=` et `<?php echo` ?

R : `<?= $x ?>` est un raccourci d'`echo`. On l'utilise pour afficher une valeur de façon concise dans le HTML.

Q : Comment gérez-vous l'affichage sur mobile ?

R : Grâce aux classes responsive de Bootstrap : `navbar-toggler` replie le menu en « hamburger », et `table-responsive` rend les tableaux défilables horizontalement.

Q : Où est gérée l'ouverture des menus déroulants ?

R : Par le JavaScript de Bootstrap (le bundle chargé en bas de page). Nous, on ne fait qu'ajouter les attributs `data-bs-toggle="dropdown"`.

Q : Y a-t-il de la logique métier dans les vues ?

R : Non. Les vues ne contiennent que des boucles d'affichage et des conditions d'affichage. Tout le métier (requêtes, calculs, paiements) est dans les contrôleurs et les modèles.